

Utilisation du contexte en React

Qu'est-ce que le contexte?

React Context nous permet de faire passer et d'utiliser (consommer) des données dans n'importe quel composant de notre application React pour lequel nous en avons besoin, et ce, sans utiliser les props.

Autrement dit, React Context permet de partager de la donnée (des states) via nos composants plus facilement.

Quand l'utiliser?

Il est pertinent d'utiliser React Context lorsque vous voulez rendre des données accessibles à l'ensemble des composants de votre application.

Ces types de données inclues :

- Données de thème (comme le mode sombre ou le mode clair)
- Données utilisateurs (l'utilisateur actuellement authentifié)
- Données spécifiques d'emplacement (comme la langue utilisateur ou les paramètres locaux)

Les données utilisées dans React Context ne doivent pas être mises à jour régulièrement.

Pourquoi? Parce que Context n'a pas été créé comme un système de gestion de state complet. Il a été créé afin de permettre de consommer plus facilement de la donnée.

Vous pouvez penser à React Context, comme un équivalent à des variables globales pour nos composants React.

Quels sont les problèmes que React Context permet de résoudre ?

React Context nous aide à éviter ce que l'on appelle le "props drilling".

Le **props drilling** est un terme utilisé afin de décrire le fait de passer des props sur plusieurs niveaux vers un composant imbriqué, à travers des composants qui n'en ont pas l'utilité.

Voici un exemple de props drilling. Dans cette application, on a accès aux données de thème que l'on veut faire passer en tant que props à tous les composants de notre application.

Cependant, comme vous pouvez le voir, les enfants directs d'App, comme Header, doivent aussi transmettre les données de thème en utilisant les props.

```
export default function App({ theme }) {  
  return (  
    <>  
      <Header theme={theme} />  
      <Main theme={theme} />  
      <Sidebar theme={theme} />  
      <Footer theme={theme} />  
    </>  
  );  
}
```

```
function Header({ theme }) {  
  return (  
    <>  
      <User theme={theme} />  
      <Login theme={theme} />  
      <Menu theme={theme} />  
    </>  
  );  
}
```

Quel est le problème avec cet exemple ?

Le problème est que l'on fait passer la prop `theme` à travers de multiples composants qui ne les utilisent pas directement.

Le composant `Header` n'a pas besoin de `theme` si ce n'est pour le transmettre à ses composants enfants. En d'autres mots, il serait préférable pour `User`, `Login` et `Menu` de consommer les données de `theme` directement.

C'est l'avantage de `React Context` – on peut contourner complètement l'utilisation des props et ainsi éviter le problème du props drilling.

Comment utiliser React Context ?

Contexte est une API, intégrée à React, depuis la version 16 de React.

Ce qui signifie que l'on peut créer et utiliser Contexte directement en important React dans n'importe quel projet React.

Il y a 4 étapes pour utiliser React Context :

1. Créer un Contexte en utilisant la méthode `createContext`.
2. Prendre le Contexte tout juste créé et utiliser le `context provider` (ou fournisseur de contexte) pour englober votre arbre de composants.
3. Mettre la valeur de votre choix dans la prop `value` de votre `context provider`.
4. Lire cette valeur dans n'importe quel composant en utilisant le `context consumer` (ou consommateur de contexte).

Liens vers la documentation officielle:

[createContext](#)

[useContext](#)

[Schémas et exemples \(Openclassrooms\)](#)